

Worksheet 2

Student Name: Saloni Kumari

Branch: MCA (General)

Semester: 3rd

Subject Name: Machine Learning Lab

UID: 25MCA20087

Section/Group: 25MCA-2A

Date of Performance: 03-08-2026

Subject Code: 25CAP-702

Aim/Overview of the practical:

To implement Linear Regression using Python and Scikit-Learn on a real-world dataset, plot the best fit regression line, and evaluate the model using MSE and RMSE.

Objectives:

1. To understand the basic concept and working principle of Linear Regression.
2. To import and preprocess a real-world dataset using Python libraries such as Pandas and NumPy.
3. To split the dataset into training and testing sets for model development.
4. To implement the Linear Regression algorithm using the Scikit-Learn library.
5. To train the regression model and make predictions on test data.
6. To visualize the relationship between the independent and dependent variables using scatter plots and the best-fit regression line.
7. To evaluate the performance of the regression model using Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

Code/Output:

```
[2]
✓ 1s # Step 1: Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

[3]
✓ 0s # Step 2: Load Dataset
# Replace "student_scores.csv" with your dataset filename
df = pd.read_csv("student-scores.csv")
```

▶ # Step 3: Display Dataset Information

```
print("First 5 Rows of Dataset:")
print(df.head())
```

```
print("\nDataset Shape:")
print(df.shape)
```

... First 5 Rows of Dataset:

	id	first_name	last_name	email	gender	\
0	1	Paul	Casey	paul.casey.1@gslingacademy.com	male	
1	2	Danielle	Sandoval	danielle.sandoval.2@gslingacademy.com	female	
2	3	Tina	Andrews	tina.andrews.3@gslingacademy.com	female	
3	4	Tara	Clark	tara.clark.4@gslingacademy.com	female	
4	5	Anthony	Campos	anthony.campos.5@gslingacademy.com	male	

	part_time_job	absence_days	extracurricular_activities	\
0	False	3	False	
1	False	2	False	
2	False	9	True	
3	False	5	False	
4	False	5	False	

	weekly_self_study_hours	career_aspiration	math_score	history_score	\
0	27	Lawyer	73	81	
1	47	Doctor	90	86	
2	13	Government Officer	81	97	
3	3	Artist	71	74	
4	10	Unknown	84	77	

	physics_score	chemistry_score	biology_score	english_score	\
0	93	97	63	80	
1	96	100	90	88	
2	95	96	65	77	
3	88	80	89	63	

4	65	65	80	74
---	----	----	----	----

	geography_score	total_score
0	87	574
1	90	640
2	94	605
3	86	551
4	76	521

Dataset Shape:
(2000, 18)

```
print("\nDataset Information:")
print(df.info())
```

...

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 18 columns):
#   Column                               Non-Null Count  Dtype
---  -
0   id                                    2000 non-null   int64
1   first_name                           2000 non-null   object
2   last_name                            2000 non-null   object
3   email                                2000 non-null   object
4   gender                               2000 non-null   object
5   part_time_job                        2000 non-null   bool
6   absence_days                         2000 non-null   int64
7   extracurricular_activities          2000 non-null   bool
8   weekly_self_study_hours              2000 non-null   int64
9   career_aspiration                   2000 non-null   object
10  math_score                           2000 non-null   int64
11  history_score                        2000 non-null   int64
12  physics_score                        2000 non-null   int64
13  chemistry_score                      2000 non-null   int64
14  biology_score                        2000 non-null   int64
15  english_score                        2000 non-null   int64
16  geography_score                      2000 non-null   int64
17  total_score                          2000 non-null   int64
dtypes: bool(2), int64(11), object(5)
memory usage: 254.0+ KB
None
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

...

```
Missing Values:
id                                     0
first_name                           0
last_name                            0
email                                0
gender                               0
part_time_job                        0
absence_days                         0
extracurricular_activities          0
weekly_self_study_hours              0
career_aspiration                   0
math_score                           0
history_score                        0
physics_score                        0
chemistry_score                      0
biology_score                        0
english_score                        0
geography_score                      0
total_score                          0
dtype: int64
```

```
# Step 4: Select Independent (X) and Dependent (y) Variables
#check the column name
print(df.columns.tolist())
```

```
['career_aspiration', 'math_score', 'history_score', 'physics_score', 'chemistry_score', 'biology_score', 'english_score', 'geography_score', 'total_score']
```

```
# Create an overall score from the available subject scores
score_columns = [
    'math_score', 'history_score', 'physics_score',
    'chemistry_score', 'biology_score', 'english_score',
    'geography_score'
]
```

```
# Average score (you can use .sum(axis=1) instead for total marks)
df['total_score'] = df[score_columns].mean(axis=1)
```

```
X = df[['weekly_self_study_hours']]
y = df['total_score']
```

```
# Step 5: Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Step 6: Create Linear Regression Model
model = LinearRegression()
```

```
# Step 7: Train the Model
model.fit(X_train, y_train)
```

```
LinearRegression
```

```
LinearRegression()
```

```
# Step 8: Predict Test Data
y_pred = model.predict(X_test)
```

```
# Step 9: Display Regression Equation
print("\nRegression Equation")
print("-----")
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
```

Regression Equation

Slope (Coefficient): 0.24279138785473026
Intercept: 76.55687163485204

```
# Step 10: Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)
print("\nMean Squared Error (MSE):", mse)
```

```
# Step 11: Calculate Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)
print("Root Mean Squared Error (RMSE):", rmse)
```

Mean Squared Error (MSE): 31.706203293476097
Root Mean Squared Error (RMSE): 5.630826164380863

```
#Step 12: Compare Actual and Predicted Values
comparison = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred
})
```

```
print("\nActual vs Predicted Values")
print(comparison)
```

Actual vs Predicted Values

	Actual	Predicted
0	90.428571	81.169908
1	79.857143	76.799663
2	76.714286	80.441534
3	74.571429	77.770829
4	76.857143	77.042454
..	...	...
395	77.142857	79.955951
396	83.000000	80.198742
397	91.142857	85.054570
398	83.000000	77.042454
399	77.857143	77.770829

[400 rows x 2 columns]

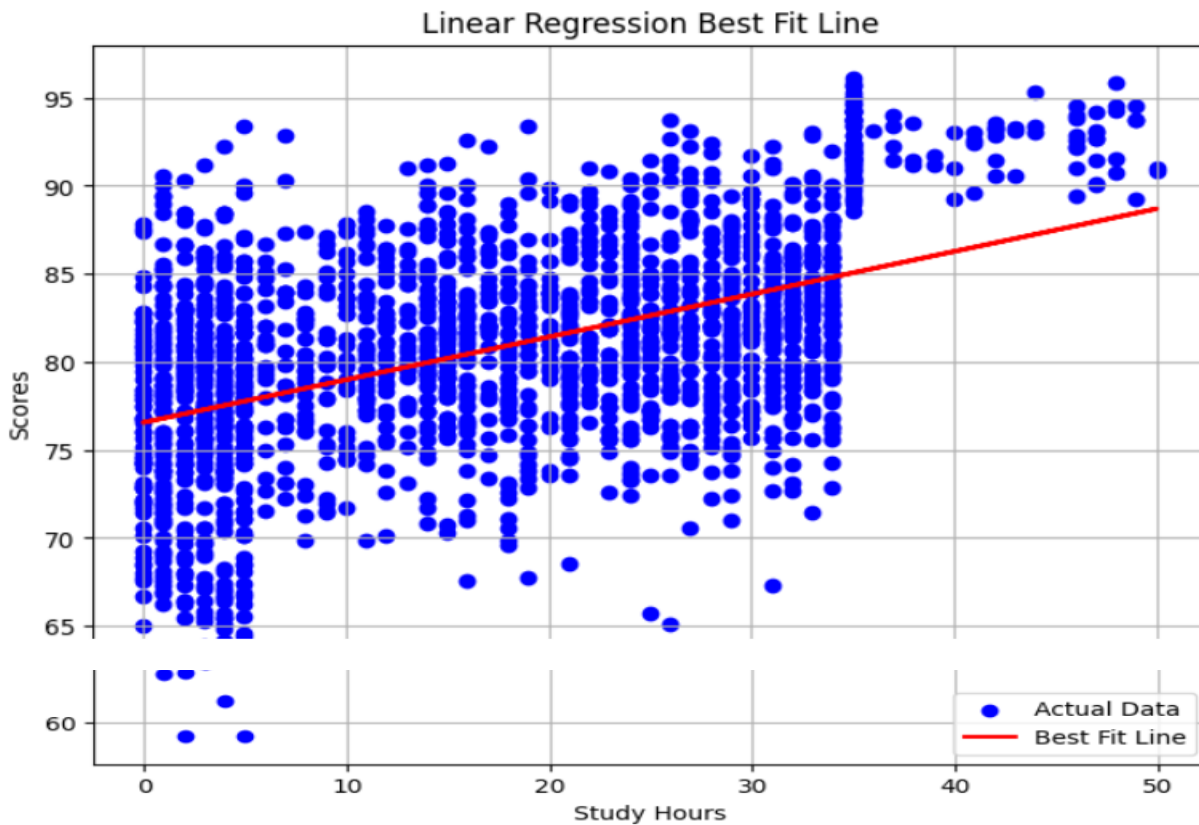
```
# Step 13: Plot Best Fit Line
plt.figure(figsize=(8,6))

plt.scatter(X, y, color='blue', label='Actual Data')

plt.plot(X, model.predict(X), color='red', linewidth=2, label='Best Fit Line')

plt.xlabel('Study Hours')
plt.ylabel('Scores')
plt.title('Linear Regression Best Fit Line')
plt.legend()
plt.grid(True)

plt.show()
```



```
df.to_csv('new_student_scores')
```

Learning Outcomes:

- Understand the concept of Linear Regression.
- Import and preprocess datasets using Python libraries.
- Implement Linear Regression using Scikit-Learn.
- Visualize the best fit regression line.
- Evaluate model performance using MSE and RMSE.
- Interpret regression model results.